A Production-Grade Linux Network Observability System

A Final Year Project Report

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology in Computer Science & Engineering

Submitted By

Anil Kumar Reddy

Under the Guidance of

Prof. [Supervisor Name]

Department of Computer Science & Engineering [Your University / Institution Name]

Bhopal, Madhya Pradesh, India

Academic Year 2024–2025

TABLE OF CONTENTS

1.1 Background and Motivation.....3
1.2 Problem Statement.....5
1.3 Objectives of the Project.....6
1.4 Scope and Limitations.....8
1.5 Organisation of the Report.....9
1.6 Significance of the Project.....9
1.7 Definitions and Abbreviations.....10
1.8 Chapter Summary.....12

2.1 Network Monitoring Approaches.....13
2.2 Kernel-Level Packet Capture Techniques.....16
2.3 DNS Analysis and Threat Detection.....18
2.4 Geographic Routing and BGP Monitoring.....20
2.5 Anomaly Detection in Network Traffic.....22
2.6 Comparative Study of Monitoring Architectures 24
2.7 Review of Qt-Based Network Tools.....26
2.8 Review of Process Attribution Techniques.....27
2.9 Summary of Related Work.....29

3.1 Identified Research Gaps.....31
3.2 Proposed System Architecture.....32
3.3 System Design and Modules.....33
3.4 Tools and Technologies Used.....35
3.5 Detailed Data Flow Diagram.....36
3.6 /proc File Interface Design.....37
3.7 Netlink Protocol Design.....40
3.8 Anomaly Detection Algorithm Design.....41
3.9 DNS Pipeline Design.....43
3.10.....Security Considerations 44
3.11.....Chapter Summary 45

4.1 Experimental Setup.....46
4.2 Connection Tracking Results.....47
4.3 Anomaly Detection Performance.....48
4.4 Route Tracing and Geo-Enrichment.....49
4.5 GUI Performance and Usability.....50
4.6 DNS Resolution Performance.....51
4.7 Memory and Resource Usage Analysis.....52
4.8 Comparison with Existing Tools.....53
4.9 Case Study 1: Detecting a Port Scan.....55
4.10.....Case Study 2: High Bandwidth Exfiltration 57
4.11.....Case Study 3: BGP Route Change Detection 59
4.12.....Limitations Observed During Testing 61

Discussion and Analysis.....62

5.1 Conclusions.....63
5.2 Future Work.....65
5.3 Lessons Learned.....67
5.4 Project Timeline.....68
5.5 Comparison with Project Objectives.....69
5.6 Final Remarks.....70

6.1 Kernel Module Packet Processing Pipeline.....71
6.2 TCP Connection State Machine.....73
6.3 DNS Parser RFC 1035 Implementation.....74
6.4 Route Store Protocol.....75
6.5 GUI ProcReader Column Mapping.....76
6.6 Netlink Event Emission.....77

7.1 Unit Testing.....78
7.2 Integration Testing.....79
7.3 Stress Testing.....80
7.4 Edge Case Analysis.....81

HAPTER 1 Introduction & Objectives

In the modern computing landscape, network connectivity is ubiquitous and indispensable. Every process on a Linux system — whether a user application, a system daemon, or a background service — routinely establishes network connections to communicate with remote hosts. While this connectivity enables powerful functionality, it also creates significant challenges for system administrators, security analysts, and developers who need to understand precisely what is happening on their machines at the network level. The sheer volume of concurrent connections observed on a modern workstation — hundreds of simultaneous TCP sessions across dozens of processes — makes manual inspection infeasible.

The Linux networking stack has evolved considerably since the original socket API was introduced. Modern kernels implement a fully asynchronous, multi-queue networking subsystem that processes packets at line rate using NAPI (New API) polling, hardware offload features such as TSO/GSO/GRO, and receive-side scaling (RSS) across CPU cores. Netfilter, introduced by Rusty Russell in 1998 and shipped as part of the mainline kernel since 2.4, provides five hook points in the IPv4 and IPv6 packet processing paths that allow kernel modules to inspect and modify every packet. These hooks form the foundation of iptables, nftables, and — crucially — KTA.

The case for operating at the kernel level rather than in userspace is compelling from both a completeness and a performance perspective. Userspace tools that read /proc/net/tcp operate on a snapshot that is already stale by the time the read completes; connections that open and close within a single polling interval are permanently lost. In contrast, a netfilter hook sees every packet before any userspace process has a chance to read a socket, providing true completeness. Furthermore, the kernel can aggregate statistics across hundreds of connections with microsecond latency that no userspace polling mechanism can match.

The evolution from BPF (Berkeley Packet Filter, 1993) to eBPF (extended BPF, introduced in kernel 3.18 in 2014) represents the state of the art in kernel-level observability. eBPF allows verified bytecode programs to be injected into the kernel at runtime, attaching to tracepoints, kprobes, and network events without requiring a kernel module. Tools such as Cilium, Falco, and bpftrace are built on eBPF. However, eBPF programs face strict limits: the verifier rejects programs with more than one million instructions (pre-kernel 5.1), loops are constrained, and state must be managed through kernel-side maps. For a complex monitoring suite that maintains connection tables, DNS caches, anomaly detectors, and route stores, a kernel module written in C offers cleaner state management, unrestricted loop constructs, and direct access to all kernel data structures — at the cost of requiring more rigorous development discipline.

The Kernel Traffic Analyzer (KTA) was conceived and implemented during the final year of a B.Tech Computer Science & Engineering programme at a time when the author was working through a 4G mobile hotspot in Bhopal, Madhya Pradesh, India. The experience of being unable to determine which applications were consuming the limited mobile data allocation, which countries the data was transiting, and

whether any anomalous activity was occurring in the background provided the immediate motivation for the project. KTA v7.0 represents

approximately twelve thousand lines of code across C, C++, and Python — the culmination of seven major development iterations — and addresses all five research gaps identified in the literature.

Despite the maturity of the Linux networking ecosystem, no single open-source tool available as of 2024 simultaneously provides kernel-level connection tracking, per-process attribution, real-time DNS correlation, geographic route visualisation, anomaly detection, and integrated remediation. This gap is not merely inconvenient — it has real security consequences. Industry reports from Verizon (DBIR 2023) indicate that 74% of breaches involved the human element, but a significant fraction of the remainder involved malware or compromised software that exfiltrated data silently over months. In such scenarios, the ability to immediately attribute every outbound byte to a specific process and destination country could dramatically reduce dwell time — the median time between breach and detection, which the 2023 IBM Cost of a Data Breach Report places at 204 days.

Consider a concrete scenario: a developer installs an npm package that includes a supply-chain attack payload. The payload opens a reverse shell to a C2 server in a foreign country. With existing tools, the developer would need to cross-reference netstat output (showing an open TCP connection), DNS resolver logs (showing a suspicious domain resolution), and router logs (showing outbound traffic) to detect the activity. KTA surfaces this in a single screen: the Connections tab shows the node process with ESTABLISHED state, the DNS Map tab shows the domain resolved seconds before the connection, the Route Map animates the hop path through foreign ASNs, and the Anomalies tab may flag HIGH_BW if data is being exfiltrated at speed.

The following six critical deficiencies in existing tools are addressed by KTA:

- **No kernel-level visibility:** Userspace tools like ss and netstat only read /proc/net/tcp and miss packets that bypass the socket layer or belong to short-lived connections. High-frequency connections that open and close within one polling cycle are silently dropped from the monitoring record entirely.
- **Lack of per-process attribution:** Raw packet capture tools such as tcpdump operate at the interface level and do not attribute traffic to specific OS processes or executable paths. An analyst cannot determine whether a connection originated from Firefox, a malicious background daemon, or a compromised library.
- **No real-time DNS correlation:** Knowing that a connection goes to 142.250.77.46 is far less meaningful than knowing it goes to google.com. DNS correlation must happen at packet interception time to be useful; post-hoc DNS resolution is unreliable because the DNS record may have been updated or the TTL may have expired before the correlation is attempted.
- **No geographic routing awareness:** Administrators cannot easily determine which countries their data transits through, nor detect unexpected routing changes that could indicate BGP hijacking or route leaks. Mobile hotspot users behind carrier-grade NAT face additional challenges because ICMP traceroute is typically blocked.

- **Fragmented anomaly detection:** Security teams rely on separate SIEM tools for anomaly detection, creating latency between an event and its detection. Kernel-level detection with

sub-second latency is achievable and eliminates the need for external tooling for common anomaly types.

- **No integrated firewall control:** Blocking a suspicious connection typically equires switching to a terminal and writing iptables rules manually, interrupting the monitoring workflow and delaying response by tens of seconds.

Seven primary objectives were defined at the outset of the project. Each objective corresponds to a specific technical gap identified in Section 1.2 and is directly traceable to an implemented subsystem in KTA.

Objective 1 — Kernel-level packet interception: Design and implement a Linux kernel module that intercepts all TCP and UDP traffic at the NF_INET_LOCAL_IN and NF_INET_LOCAL_OUT netfilter hooks without dropping any packets. The hooks must operate at NF_IP_PRI_FIRST priority to ensure they run before connection tracking, and must return NF_ACCEPT unconditionally so that network functionality is not impaired. This objective establishes the foundation upon which all other features are built.

Objective 2 — Per-process attribution: Implement per-process attribution by correlating kernel sockets to PIDs using a multi-level cache hierarchy (sock_cache, inode_cache, flow_cache, full resolver). The design rationale for the four-level hierarchy is to minimise the frequency of expensive for_each_process() scans: sock_cache hits take O(1) time, inode and flow cache hits take O(log n), and only the full resolver requires walking the process list. The target attribution accuracy is above 95% for TCP connections with duration exceeding 500 ms.

Objective 3 — In-kernel DNS parsing: Parse DNS responses in RFC 1035 wire format within the netfilter hook to build a real-time IP domain mapping cache available at connection creation time. The design rationale is that DNS responses always precede the TCP SYN that opens the connection; by parsing the DNS response in the incoming hook (NF_INET_LOCAL_IN) before parse_packet() runs, the domain name is inserted into dns_map before the connection entry is created.

Objective 4 — Route daemon with geo-enrichment: Integrate a Python-based route daemon that performs TCP traceroute on port 443 and enriches each hop with MaxMind GeoLite2 city, country, latitude/longitude, ASN, and organisation data. The use of TCP mode (traceroute -T) rather than ICMP is motivated by the deployment environment: 4G NAT typically blocks ICMP but passes TCP SYN packets on port 443 to enable legitimate HTTPS traffic.

Objective 5 — Anomaly detection: Implement five lightweight anomaly detection algorithms (port scan, SYN flood, connection burst, high connection count, high bandwidth) operating entirely within the kernel using per-process velocity trackers. The kernel-only design ensures detection latency is bounded by the 1-second rate window rather than by any IPC or polling round-trip.

Objective 6 — Proc interface and Qt6 GUI: Expose all collected data through seven /proc file interfaces consumed by a Qt6 GUI refreshing every second across 15 specialised monitoring tabs. The proc-file contract decouples the kernel module from the GUI, allowing the GUI to be updated or replaced without modifying kernel code.

Objective 7 — Integrated remediation: Provide one-click firewall control (iptables), bandwidth throttling (tc-netem), data export to JSON/CSV/PDF, and a system tray with anomaly badge

notifications. The design rationale is that monitoring without the ability to act is incomplete; a network analyst should be able to respond to a detected threat within seconds without leaving the KTA interface.

The scope of this project covers IPv4 and partial IPv6 (last 32 bits stored) TCP and UDP traffic on Ubuntu 22.04 LTS with Linux kernel 6.8.0. The system monitors the local machine only and does not perform deep packet inspection beyond DNS response parsing. The firewall and throttling features apply to OUTPUT traffic via iptables and tc-netem respectively.

The following limitations were accepted as part of the project scope:

- Full IPv6 128-bit address tracking is not implemented. The flow_key struct stores only 32 bits of the IPv6 source and destination address. This is flagged as future work and does not affect IPv4 monitoring, which is the primary use case.
- Route tracing relies on traceroute which requires root privileges. On networks where even TCP SYN on port 443 is rate-limited or blocked, hop discovery may be incomplete.
- The connection table is capped at 2,048 entries. Under sustained high-traffic conditions (observed peak: 1,847 entries), older CLOSED entries are evicted to make room for new connections.
- MaxMind GeoLite2 databases require a free account registration and periodic manual updates. The accuracy of city-level geolocation varies by region; ASN-level accuracy is typically above 99%.
- The SparklineDelegate in the Qt6 GUI must not inherit QObject (no Q_OBJECT macro) due to a vtable linker conflict with QStyledItemDelegate. This is a known Qt limitation that prevents using signals/slots directly in sparkline rendering.
- The module version string in the kernel module reports 7.0 but the README in the GitHub repository still reads 6.0 due to documentation not being updated after the final version increment.

The remainder of this report is structured as follows. Chapter 2 presents a comprehensive literature survey of existing network monitoring techniques, kernel-level packet capture approaches, DNS analysis methods, geographic routing tools, and anomaly detection frameworks. Chapter 3 identifies the research gaps and presents the proposed system architecture with detailed module design. Chapter 4 presents experimental results. Chapter 5 draws conclusions and outlines future directions. Chapter 6 provides implementation details for key subsystems. Chapter 7 describes the testing and validation approach. References and appendices follow.

KTA represents a contribution to the open-source network security and observability ecosystem that is distinctive in its combination of depth and breadth. While individual capabilities such as kernel packet capture or DNS monitoring exist in separate tools, no comparable open-source project integrates all of them in a single installable kernel module with a production-quality GUI. The project demonstrates that a single developer, working within the constraints of a final-year B.Tech project timeline, can build a system of this complexity using freely

available tools and

publicly documented kernel APIs.

From an educational perspective, KTA serves as a reference implementation for several advanced Linux kernel programming techniques: softirq-safe spinlock usage (spin_lock_bh), RCU read-lock protected task structure access, GFP_ATOMIC memory allocation in interrupt context, workqueue-based deferred work scheduling, and /proc filesystem virtual file implementation. These topics are typically covered only in advanced kernel development courses and are rarely demonstrated in conjunction with a complete application-level consumer (the Qt6 GUI).

From a practical security perspective, KTA addresses a real need in the Indian developer and SME community where dedicated network monitoring appliances are cost-prohibitive. A free, open-source tool that runs on commodity hardware with a 4G internet connection provides visibility that was previously available only to enterprises with dedicated SOC (Security Operations Centre) infrastructure.

The following table defines key terms and abbreviations used throughout this report.

Term / Abbreviation	Definition
ASN	Autonomous System Number — a globally unique identifier assigned to a collection of IP ad
BPF / eBPF	Berkeley Packet Filter / extended BPF — a kernel virtual machine that executes sandboxed
BGP	Border Gateway Protocol — the inter-domain routing protocol of the internet, used by ASNs
CLOSED	TCP connection state after both FIN/ACK exchange is complete and the connection entry is
DNS	Domain Name System — the hierarchical naming system that maps human-readable domai
FIN_WAIT	TCP state after the local endpoint has sent FIN, awaiting the remote FIN+ACK.
flow_key	KTA internal struct: {src_ip, dst_ip, src_port, dst_port, proto} identifying a unique 4-tuple afte
GFP_ATOMIC	Linux kernel memory allocation flag meaning "do not sleep" — required for all allocations in
GeoLite2	MaxMind's free-tier IP geolocation database providing city, country, and ASN data.
GUI	Graphical User Interface — in KTA context, the Qt6 C++ application.
inode_cache	KTA second-level cache mapping socket inode numbers to PIDs.
iptables	Linux userspace firewall administration tool that manages netfilter rules.
KTA	Kernel Traffic Analyzer — the system described in this report.
NF_ACCEPT	Netfilter verdict meaning "continue normal packet processing" — returned by all KTA hooks.
NF_INET_LOCAL_IN	Netfilter hook point for packets destined to the local machine (post-routing).
NF_INET_LOCAL_OUT	Netfilter hook point for packets generated by local processes (pre-routing).
NF_IP_PRI_FIRST	Highest priority value (-300) for netfilter hook registration — ensures KTA runs before conntr

dress ranges
u programs
in ke to
advertise IP
eligible for
evic n names
to IP

r
make_canoni
softirq/interru
pt

ack.

Term / Abbreviation	Definition
RFC 1035	IETF standard defining the DNS wire format for queries and responses.
RCU	Read-Copy-Update — a synchronisation mechanism for read-mostly data structures in the kernel.
skb	Socket buffer (struct sk_buff) — the primary kernel data structure for network packets.
sock_cache	KTA first-level cache mapping struct sock pointers directly to PIDs.
SYN_SENT	TCP state after the client has sent SYN, before receiving SYN+ACK.
tc / tc-netem	Linux Traffic Control — kernel subsystem for packet scheduling, shaping, and emulation.
TTL	Time To Live — a field in IP headers limiting hop count; also used in DNS records cache to specify lifetime.
UDP	User Datagram Protocol — connectionless transport protocol used for DNS, streaming, and other latency-s ocess context).

Table 1.1 — Definitions and Abbreviations

This chapter established the motivation for KTA by highlighting the limitations of existing network monitoring tools and the real security implications of inadequate network visibility. The problem statement was articulated as six specific deficiencies in existing tools. Seven project objectives were defined, each with a clear design rationale traceable to an implemented subsystem. The scope and limitations were defined honestly, including incomplete IPv6 support and the 2,048-connection table cap. The significance of the project for education, security, and the open-source community was discussed. Chapter 2 now surveys the existing literature to establish the academic context for these contributions.

CHAPTER 2 Literature Survey

This chapter surveys existing work in the areas of network traffic monitoring, kernel-level packet capture, DNS-based analysis, geographic routing, anomaly detection, and related GUI frameworks. The survey establishes the academic and technical context for the research gaps addressed by KTA and justifies the design decisions made in Chapter 3.

2.1.1 Traditional Userspace Tools

tcpdump, originally developed by Van Jacobson, Craig Leres, and Steven McCanne at Lawrence Berkeley National Laboratory in 1989, pioneered kernel-assisted packet capture through the Berkeley Packet Filter mechanism. BPF allows filter expressions to be compiled into a bytecode program that runs in kernel space to decide which packets to copy to userspace, dramatically reducing the overhead compared to copying all packets and filtering in userspace. tcpdump remains the de facto standard for command-line packet capture and is pre-installed on virtually every Linux distribution. However, it does not associate packets with processes and requires substantial manual effort to correlate output with application behaviour. It also writes raw PCAP files that require separate tools for analysis.

Wireshark (originally Ethereal, renamed in 2006) was created by Gerald Combs in 1998 and has since grown into the most widely used graphical packet analyser in the world. Wireshark dissects over 3,000 protocols, supports live capture via pcap and libpcap, and provides a powerful display filter language. The application is invaluable for protocol debugging and forensic analysis of PCAP recordings. However, it is not designed for continuous production monitoring: live capture at high packet rates consumes substantial CPU and memory, and the interface becomes unwieldy when thousands of connections are open simultaneously. Critically, Wireshark does not attribute packets to OS processes natively, though the recent addition of process information via extcap interfaces partially addresses this.

ss (socket statistics) and its predecessor netstat are included in every major Linux distribution. ss reads from kernel socket diag infrastructure (SOCK_DIAG, RFC 7395) which provides more detailed information than /proc/net/tcp but still operates on a point-in-time snapshot. The -p flag displays the PID and process name for each socket by cross-referencing /proc/[pid]/fd. The fundamental limitation is that ss shows only currently open connections; any connection that opened and closed between two ss invocations is invisible. Under typical workloads with HTTP/2 connection multiplexing and short-lived HTTPS connections, this means a significant fraction of network activity is unobserved.

nethogs, written by Arnout Engelen (originally Rob van Hoecke) around 2004, attempts to provide per-process bandwidth monitoring by parsing /proc/net/tcp periodically and correlating with socket inodes from /proc/[pid]/fd. It displays a live table of per-process bandwidth sorted by current usage. nethogs is lightweight and useful for quick bandwidth attribution but suffers from:
(a) the same snapshot limitation as netstat; (b) inability to show connection destinations or DNS names; (c) no anomaly detection; (d) purely text-based interface with no history or export capability.

iftop (by Paul Warren, 2002) monitors traffic on a single network interface and displays bandwidth usage by host pair in a ncurses interface. It operates at the packet level using libpcap and can show DNS resolution for endpoints. However, it provides no per-process

attribution, no anomaly detection, no route tracing, and no persistent history. It is essentially a bandwidth monitor rather than a security-oriented observability tool.

2.1.2 Flow-Based and eBPF Monitoring

NetFlow, developed by Cisco in 1996, exports flow summaries from routers and switches to collection servers (NetFlow collectors). A flow is defined as a unidirectional sequence of packets sharing a 5-tuple (source/destination IP, source/destination port, protocol). NetFlow v9 and its open successor IPFIX (RFC 7011, 2013) add flexible field definitions and template support. NetFlow provides excellent scalability for network-wide visibility but operates at the network device level and is unavailable for monitoring an individual host's internal processes without router-side configuration. Furthermore, NetFlow discards payload content and does not attribute flows to OS processes.

sFlow (RFC 3176) takes a sampling approach, forwarding one in N packets to a collector. This reduces overhead at the cost of accuracy, making it unsuitable for security monitoring where every connection matters. For a local host monitor like KTA, sampling is not acceptable.

Extended Berkeley Packet Filter (eBPF), mainlined in kernel 3.18 (2014) with major enhancements in 4.4 (maps), 4.7 (tc/cls_bpf), and subsequent releases, has emerged as the dominant paradigm for kernel-level observability without writing a traditional kernel module. The eBPF verifier statically analyses bytecode for safety (no infinite loops, no invalid memory accesses, bounded stack usage) before loading it into the kernel. eBPF programs can attach to tracepoints (kprobes, uprobes, tracepoints, perf events) and network events (tc, XDP, socket filters). Tools built on eBPF include Cilium (network policy and observability for Kubernetes), Falco (security monitoring), Pixie (application performance monitoring), and bpftrace (DTrace-style tracing language).

Despite its power, eBPF has limitations relevant to KTA's design. The complexity limit (1 million instructions pre-5.1, raised to 2 million in 5.12+) constrains the amount of per-packet processing that can be performed in a single program. Maintaining a connection table of 2,048 entries with DNS caches and anomaly counters requires careful partitioning across multiple eBPF maps and programs, making the resulting system harder to reason about than a single-file C implementation. Additionally, eBPF requires BPF Type Format (BTF) metadata for CO-RE (Compile Once - Run Everywhere) portability, adding toolchain complexity. For an academic project with a fixed kernel version target (6.8.0), a kernel module offers a simpler and more direct path to the required functionality.

The Linux Netfilter framework, introduced by Rusty Russell in 1998 for the 2.4 kernel series, provides five hook points in the IPv4 packet processing path: NF_INET_PRE_ROUTING (before routing decision), NF_INET_LOCAL_IN (for locally delivered packets), NF_INET_FORWARD (for forwarded packets), NF_INET_LOCAL_OUT (for locally generated packets), and NF_INET_POST_ROUTING (after routing, before transmission). IPv6 has an identical set of hooks. KTA registers at LOCAL_IN and LOCAL_OUT to capture all traffic to and from the local machine — the correct position for a per-host monitor.

Hook registration uses nf_hook_ops structs specifying the hook number, priority,

protocol family (NFPROTO_IPV4), and callback function pointer. KTA registers at NF_IP_PRI_FIRST (-300), which is lower than the connection tracking priority (NF_IP_PRI_CONNTRACK = -200)

and lower than NAT (-100). This ensures KTA sees the original pre-NAT packet, which is essential for correct IP-to-process attribution.

PID attribution from a netfilter hook presents a significant challenge: the hook runs in softirq context, where current may or may not point to the process that generated the packet. For outgoing packets (LOCAL_OUT), current is typically the process that called send() or write() on the socket, making direct current->pid attribution feasible. For incoming packets (LOCAL_IN), current is whatever happened to be scheduled when the packet arrived, making direct attribution impossible. KTA handles this asymmetry by: (a) using direct current->pid for outgoing packets when the socket is accessible; (b) relying on the cache hierarchy for all other cases, with async workqueue resolution for cases that miss all caches.

The sock_cache is a hash table mapping struct sock* pointers (obtained from the skb's sk field) to PIDs. This cache is populated by the workqueue resolver and hits on subsequent packets from the same socket. The inode_cache maps socket inode numbers (from sock->sk_socket -> file->f_inode->i_ino) to PIDs and is populated by scanning /proc/[pid]/fd entries in the workqueue. The flow_cache maps the 5-tuple flow_key to PIDs and acts as a fallback for UDP flows where the socket may be reused across connections. The full resolver is an expensive for_each_process() scan that walks every process and every file descriptor, comparing inode numbers; it is invoked only when all caches miss and runs asynchronously in the ta_resolver workqueue to avoid blocking the softirq path.

A critical implementation detail is the use of rcu_dereference(mm->exe_file) to obtain the executable file path for a process. In Linux kernel versions prior to 6.7, this was done via get_mm_exe_file(). That function was removed in kernel 6.7 (commit 47b9f88); KTA uses the RCU-protected direct access pattern, holding the RCU read lock with rcu_read_lock() around the dereference. This is the correct and forward-compatible approach.

Stack frame discipline is critical in netfilter hooks. The Linux kernel enforces an 8 KB kernel stack per task (historically 4 KB on 32-bit systems, configurable on 64-bit). KTA's packet processing functions keep all local variables in a single combined frame under 1,024 bytes, with larger structures allocated on the heap using GFP_ATOMIC kmalloc. This avoids stack overflow in deeply nested call chains (netfilter hook packet_parser stats_update anomaly detection).

DNS is the foundational naming layer of the Internet and has been the subject of extensive security research. Paul Mockapetris designed DNS in RFC 882 (1983) and the final wire format specification in RFC 1035 (1987). DNS queries and responses use a compact binary encoding that allows domain names to be compressed using pointer references within the message. Understanding this format — including pointer compression, label encoding, and the structure of resource records — is essential for implementing a correct in-kernel parser.

Passive DNS collection, pioneered by Florian Weimer at the Computer Emergency Response Team (CERT/Bund) and presented at the FIRST conference in 2005, records all DNS responses observed on a network segment and builds a historical database of IP-to-domain mappings. This database is used for threat intelligence: by querying historical DNS, analysts can determine what domains a known-malicious IP served in the past, enabling retrospective investigation of incidents.

Services like PassiveDNS, DNSDB (Farsight Security), and VirusTotal's passive DNS API implement this approach at internet scale. KTA's dns_map

implements a lightweight single-host version of this concept.

DNS-based exfiltration is a well-documented attack technique in which an attacker encodes data in DNS query labels (e.g., base32-encoded payload as a subdomain of an attacker-controlled domain) to bypass firewalls that allow DNS traffic but block direct TCP connections. Detection approaches include monitoring query volume per domain, label length distribution analysis, and entropy measurement of query names. While KTA's current implementation does not implement DNS exfiltration detection, the dns_map and DNS leak detection subsystems provide the foundation for adding it.

Pi-hole (Jacob Salmela, 2014) and AdGuard Home intercept DNS queries at the local resolver level, blocking known advertising and tracking domains by returning NXDOMAIN or a loopback address. These tools are widely deployed but monitor only queries, not the TCP connections that follow. They cannot determine which application made a given DNS query (the query comes from the local resolver's PID, not the originating application) and they do not correlate DNS resolutions with subsequent network flows. KTA takes a complementary approach: rather than blocking at the resolver, it intercepts responses at the packet level and uses the resolved domain to enrich the connection entry in the traffic table.

DNS leak detection — verifying that all DNS queries go through the configured resolver and not to external servers — addresses a common privacy concern for VPN users. When a VPN client fails to route DNS traffic through the VPN tunnel, DNS queries are visible to the ISP (DNS leak). KTA's DnsLeakTab identifies all DNS packets (UDP port 53 or TCP port 53) observed by the LOCAL_OUT hook and compares the destination IPs against the configured resolvers in /etc/resolv.conf. Any DNS query to an unexpected server is flagged as a potential leak.

DNSSEC (RFC 4033, 2005) provides cryptographic authentication of DNS responses, preventing cache poisoning attacks. While KTA does not implement DNSSEC validation (which would require a full DNSSEC resolver in the kernel — an unreasonable scope for this project), future versions could integrate with the systemd-resolved stub resolver to obtain DNSSEC validation status for each resolved domain.

MaxMind was founded in 2002 and has since become the dominant provider of IP geolocation data. The GeoLite2 databases (City and ASN variants) are freely available under a Creative Commons Attribution-ShareAlike 4.0 licence and are used in CDN routing, fraud detection, content licensing, and compliance monitoring by thousands of companies worldwide. The GeoLite2-City database maps IPv4 and IPv6 addresses to city, country, ISO country code, latitude, and longitude with an accuracy of approximately 55–80% at the city level depending on region (higher for North America and Western Europe, lower for developing regions). ASN accuracy is above 99% for publicly routable addresses. KTA stores latitude and longitude as integers multiplied by 10^6 (E6 format) to avoid floating-point in kernel structures.

The traceroute utility, written by Van Jacobson in 1987, discovers the hop-by-hop path between two hosts by sending a series of probes with increasing TTL values. When a router decrements the TTL to zero, it returns an ICMP Time Exceeded

message, revealing its IP address and round-trip time. Traditional UDP traceroute uses unprivileged UDP probes on ports above 33434; ICMP traceroute (Paris-traceroute) sends ICMP Echo probes. Both modes are frequently blocked by firewalls and NAT devices. TCP traceroute (traceroute -T) sends TCP SYN packets on a specified port and relies on the same TTL mechanism. Port 443 (HTTPS) is

chosen by KTA because it is the least likely to be blocked on any modern network.

The route daemon in KTA implements the following protocol for communicating with the kernel module: reading /proc/traffic_analyzer_routes_pending returns a list of destination IPs whose routes are needed; the read atomically transitions them from PENDING to RUNNING state, preventing duplicate concurrent traces. For each IP, the daemon runs traceroute and writes results back as DEST/STATUS/HOP lines to /proc/traffic_analyzer_routes. The kernel module parses these writes in proc_interface.c and stores results in the route_store jhash table, making them available to the GUI via the /proc/traffic_analyzer_routes read handler.

BGP hijacking represents one of the most serious threats to internet routing integrity. In a hijacking event, an ASN announces IP prefixes it does not legitimately control, causing traffic to those prefixes to be routed through the attacker's infrastructure. Notable historical incidents include the Pakistan Telecom hijacking of YouTube's prefix in 2008 (affecting global YouTube access for approximately two hours) and the Rostelecom BGP incident in 2020 that affected routes for major CDN providers. The RPKI framework (RFC 6480) provides cryptographic attestation of route origin through Route Origin Authorizations (ROAs), allowing BGP routers to reject unauthorised announcements.

KTA's BgpMonitor implements a lightweight endpoint-perspective BGP change detector: it learns a baseline {domain (ASN, country)} mapping for each monitored domain over a sliding 7-day window. When new route tracing results show a different ASN or country for a previously monitored domain, an alert is generated. This approach cannot detect short-lived hijacks that resolve before the next traceroute but provides a practical first line of defence for the most egregious and persistent routing anomalies.

Network anomaly detection has been studied since Denning's seminal 1987 paper on intrusion detection systems, which introduced the concept of statistical profiling of user and system behaviour as a basis for detecting deviations. The field has since divided into three main approaches: statistical (threshold-based), machine learning, and signature-based detection.

Threshold-based detection, while conceptually simple, remains widely used in production systems because it is computationally efficient, deterministic, and easy to tune. Enterprise tools such as Cisco NetFlow-based anomaly detectors, Arbor Networks' Peakflow, and open-source tools like Suricata's threshold engine all implement variants of rate-based thresholds for port scan, flood, and volume anomalies. KTA implements five such detectors entirely within the kernel, operating on per-process velocity counters updated on every packet.

Port scan detection: The classic threshold approach described by Staniford, Hoagland, and McAlerney in their 2002 paper uses a sliding window over the number of unique destination ports contacted by a single source. A threshold of 15 unique destination ports per second (ANOMALY_PORT_SCAN_PORTS = 15 in KTA) is consistent with conservative enterprise policies. nmap's default SYN scan rate contacts approximately 1,000 ports per second, making this threshold trivially detectable within the first second of a scan. Stealth scans (nmap -T1, -T0) operate at rates below the detection threshold and would require a longer temporal window to detect reliably.

SYN flood detection: A SYN flood is a denial-of-service attack in which the attacker sends large numbers of TCP SYN packets without completing the three-way handshake, exhausting

the target's SYN backlog. Detecting an outgoing SYN flood (a compromised host participating in a DDoS attack) requires monitoring the ratio of SYN-pending connections to total TCP connections per process. KTA flags a process when this ratio exceeds 80% (ANOMALY_SYN_RATIO = 0.80). The SYN cookie paper by Bernstein (1996) describes the defensive technique that mitigates incoming SYN floods; KTA addresses the outgoing direction.

Connection burst and high connection count detection: Bot-like or scripted network activity is characterised by opening many connections in a short period. KTA flags processes that open more than 20 new connections per second (ANOMALY_CONN_BURST = 20) or maintain more than 200 simultaneous connections (ANOMALY_HIGH_CONNS = 200). These thresholds are calibrated to allow normal browser behaviour (a page load may open 6–15 parallel connections) while flagging clearly automated activity.

High bandwidth detection: The high bandwidth anomaly (ANOMALY_HIGH_BW = 10 MB/s combined in/out rate) serves as an exfiltration indicator. A 10 MB/s threshold corresponds to 87 GB/day — well above normal workstation usage but below the sustained throughput of a network backup or large file transfer that a user would typically be aware of. The threshold is configurable per deployment.

Machine learning approaches to network anomaly detection include Isolation Forest (Liu et al., 2008), which identifies anomalies as data points that are easier to isolate in a random partition of the feature space, and LSTM-based models that learn temporal patterns in network flows (Mirsky et al., 2018 — Kitsune). These approaches require training data and periodic model updates but achieve lower false positive rates for novel attack patterns. KTA's Section 5.2 identifies ML-based anomaly detection as a priority future work item.

Network monitoring tools can be classified along three architectural dimensions: capture point (kernel vs. userspace), data persistence (stateful vs. stateless), and presentation (CLI vs. GUI vs. API). The following analysis examines how representative tools in each category compare with KTA.

Architecture	Examples	Capture Point	State	Presentation	Process Attributio
Packet sniffer	tcpdump, tshark	Kernel (BPF)	Stateless (PC	ACPL)I	No
GUI sniffer	Wireshark	Kernel (BPF)	Stateless (PC	AGPU)I	No
Proc poller	netstat, ss, nethogs	Userspace	Snapshot	CLI	Limited
Flow exporter	NetFlow, sFlow	Network device	Stateful	Collector	No
eBPF tracer	Cilium, Falco, Pixie	Kernel (eBPF)	Stateful (map	sA) PI/CLI	Yes
Kernel module	KTA (this work)	Kernel (Netfilter)	Stateful (hash	GtaUbIle+s)/ proc	Yes (4-level)

Table 2.1 — Comparative study of monitoring architectures

The kernel module architecture offers the most complete capture coverage and richest state management, at the cost of requiring root privileges for loading and a specific kernel version target. The eBPF architecture is more portable but imposes complexity limits that constrain per-packet processing. Proc-poller architectures

are safe and portable but fundamentally incomplete due to sampling gaps.

The Qt framework, now maintained by The Qt Company and available under LGPL/GPL licences, has a long history of use in network monitoring tools. Qt's signal-slot mechanism, model-view-delegate architecture, and cross-platform widget set make it well-suited for data-rich GUIs that update in real time.

Wireshark's GUI is built on Qt (migrated from GTK+ in version 1.10, 2013). The Wireshark codebase provides extensive examples of Qt best practices for network tools, including the use of QAbstractTableModel for efficiently updating large packet lists, QSortFilterProxyModel for filtering, and QStyledItemDelegate for custom cell rendering. KTA's TrafficModel and ProcModel follow this pattern directly.

NetSpeedMonitor, NetWorx, and GlassWire are commercial Qt-based network monitoring tools for Windows that provide per-process bandwidth tracking, usage history, and alert capabilities. GlassWire in particular offers a timeline-based usage history graph and network activity alerts that informed KTA's HistoryTab and AlertPopup design. However, all three tools are closed-source, Windows-only, and do not provide kernel-level capture or route visualisation.

EtherApe (a graphical network monitor for Linux) uses GTK+ and displays network activity as an animated graph with nodes representing hosts. It provides a compelling visual representation of network topology but lacks process attribution, DNS mapping, and anomaly detection. Its development appears to have stalled around 2016.

KTA's Qt6 GUI makes several design choices informed by the above survey: (a) QStackedWidget with a collapsible sidebar rather than a tabbed interface avoids the visual clutter of 15 visible tab labels; (b) QAbstractTableModel rather than QTableWidget enables O(1) data model updates without rebuilding the widget; (c) QPainter-based sparklines embedded in the table via QStyledItemDelegate provide at-a-glance rate trends without requiring a separate chart widget per row; (d) the dark theme (#060b10 background) is chosen both for aesthetics and to reduce eye strain during extended monitoring sessions.

Attributing kernel-level network events to userspace processes is a non-trivial problem that has been studied in several academic and practitioner contexts. The core challenge is that the kernel's networking stack processes packets in softirq context, where there is no guaranteed association between the current task and the packet being processed.

The Linux kernel's SOCK_DIAG interface (implemented in net/core/sock_diag.c) provides a netlink-based API for querying socket information including inode numbers. Userspace tools use this to build a {inode PID} mapping by cross-referencing with /proc/[pid]/fd entries. This approach is used by ss, nethogs, and wireshark's extcap interface. The limitation is that it is a polling-based approach with inherent race conditions: the PID table may be stale by the time the correlation is performed.

eBPF-based attribution using kprobes on tcp_v4_connect, inet_csk_accept, and udp_sendmsg system calls provides exact attribution by capturing the current task_struct at the moment of socket creation or first use. This is the approach used

by Cilium and Pixie. The limitation is that it requires separate kprobe programs for each socket operation and does not naturally handle the case where a socket is created in one thread and used in another (common in thread pools).

KTA's four-level cache hierarchy represents a pragmatic compromise: the sock_cache provides O(1) lookup for sockets that have already been resolved (71.2% hit rate in testing), the inode and flow caches handle moderately frequent lookups, and the full resolver is triggered only when all caches miss (1.1% of connections). The 4 workqueue threads (ta_resolver) ensure that resolver work does not back up under high connection rates. The 96.8% overall attribution accuracy achieved compares favourably with nethogs (which can miss attribution for short-lived connections entirely) and approaches the theoretical maximum given the latency constraints of softirq context.

The survey conducted in this chapter reveals that while each individual capability addressed by KTA — kernel packet capture, per-process attribution, DNS correlation, geographic routing, anomaly detection, and integrated remediation — exists in one or more existing tools, no single open-source tool integrates all of them. The following summary table consolidates the comparative analysis.

Tool	Kernel Capture	Per-Process	DNS Corr.	Geo Route	Anomaly	Firewall	GUI
tcpdump	Yes	No	No	No	No	No	No
Wireshark	Yes	Partial	Yes	No	No	No	Yes
netstat/ss	No	Yes	No	No	No	No	No
nethogs	No	Yes	No	No	No	No	No
iftop	Yes (BPF)	No	Yes	No	No	No	Ncurses
Cilium/eBPF	Yes	Yes	Yes	No	Yes	Yes	API only
Falco	Yes	Yes	No	No	Yes	No	No
GlassWire	Yes (Win)	Yes	Yes	No	Limited	Limited	Yes
KTA (Ours)	Yes	Yes	Yes	Yes	Yes	Yes	Yes

Table 2.2 — Comprehensive feature comparison of network monitoring tools

Based on this survey, it is clear that KTA occupies a unique position in the tool landscape as the only open-source system providing all seven capabilities simultaneously. The architectural choice of a kernel module (rather than eBPF) enables the richest possible per-packet state management on the target platform. Chapter 3 uses this foundation to present the proposed system architecture and detailed design.

CHAPTER 3 Research Gap & Proposed System

Based on the literature survey presented in Chapter 2, five research gaps are identified that motivate the design of KTA:

- **Gap 1 — Unified real-time enrichment:** No existing open-source tool provides simultaneous kernel-level capture, per-process attribution, DNS correlation, and geographic enrichment in a single pipeline with sub-second latency.
- **Gap 2 — In-kernel DNS parsing:** Existing tools either capture DNS at the resolver level (missing rogue direct DNS queries) or post-process PCAP files. Parsing DNS responses in the netfilter hook before connection tracking runs ensures immediate domain availability at connection creation time.
- **Gap 3 — TCP traceroute integration:** No monitoring GUI performs live TCP-mode traceroute to each new destination and renders the hop path geographically. Carrier-grade NAT in 4G environments blocks ICMP, making TCP mode on port 443 essential.
- **Gap 4 — Push-based netlink event stream:** Poll-based /proc reading creates up to 1-second latency and unnecessary CPU overhead. A push-based netlink multicast channel enables real-time GUI updates on connection state changes and anomaly events.
- **Gap 5 — Integrated remediation:** No existing monitoring tool integrates firewall rule management and bandwidth throttling within the monitoring interface itself.

KTA is structured as three cooperating layers. Each layer has clearly defined responsibilities and communicates with adjacent layers through well-defined interfaces.

Layer	Technology	LOC	Interface	Responsibility
Layer 1: Kernel Module	C (GCC)	~3,500	7 /proc files + netlink multica	sPtacket capture, PID attribution, DN
Layer 2: Route Daemon	Python 3	~330	/proc/ traffic_analyzer_route	sTCP traceroute, MaxMind GeoLite2
Layer 3: Qt6 GUI	C++ (Qt6)	~8,000	Reads /proc files, s1t5o mneotnliintokring tabs,	

Table 3.1 — KTA three-layer architecture summary

The three-layer design enforces a clean separation of concerns: the kernel module is responsible only for packet-level data collection and exposure through the proc interface contract; the route daemon is responsible only for traceroute and geo-enrichment; and the GUI is responsible only for presentation and user interaction. This design allows each layer to be updated independently — for example, the GUI can be replaced with a web dashboard without modifying any kernel code, as long as the proc interface contract is respected.

3.3.1 Kernel Module Subsystems

The kernel module is decomposed into 12 source files, each with a single well-

S parsing,

ano

enrichment,

h

defined responsibility. This single-responsibility design facilitates testing, code review, and

future modification.

Source File	Lines (approx.)	Responsibility
module_main.c	120	Module entry/exit, initialises subsystems in dependency order
netfilter_hook.c	180	Two nf_hook_ops, DNS interception before parse_packet()
packet_parser.c	450	10-step packet processing pipeline, PID attribution waterfall
stats.c	520	TCP FSM, rate window, velocity tracker, anomaly detection, netlink thr
proc_interface.c	380	7 /proc file creation, unified write handler, control commands
dns_parser.c	200	RFC 1035 wire format parser, pointer compression, A record extraction
dns_map.c	150	IP domain hash cache, TTL tracking, LRU eviction
route_store.c	280	jhash table for hop data, pending_write_ip state machine
resolver.c	220	ta_resolver workqueue (4 threads), socket and exe path resolver
sock_cache.c	160	struct sock* PID cache, RCU-safe operations
inode_cache.c	140	Socket inode PID cache
flow_cache.c	160	5-tuple flow_key PID cache
netlink_comm.c	190	NETLINK_USERSOCK proto 2, 4 multicast groups, 6 message

Table 3.2 — Kernel module source files and responsibilities

3.3.2 Route Daemon Design

The Python route daemon (ta_route_daemon.py, approximately 330 lines) bridges the kernel module and the MaxMind databases. It operates as a long-running system service and uses no external dependencies beyond the geoip2 Python package and the standard traceroute utility. The daemon opens the GeoLite2-City and GeoLite2-ASN database files once at startup using the geoip2.database.Reader context manager, avoiding the overhead of repeated file opens.

The main loop polls /proc/traffic_analyzer_routes_pending every 2 seconds. Reading this file atomically transitions all PENDING entries to RUNNING state in the kernel module (implemented via a single mutex-protected operation in proc_interface.c), preventing duplicate work if the daemon restarts or if multiple daemon instances are accidentally run. For each IP, the daemon checks the in_progress set before launching a traceroute subprocess; this set prevents concurrent traces for the same destination IP.

3.3.3 GUI Architecture

The Qt6 GUI (approximately 8,000 lines of C++) is structured around a QMainWindow with a QStackedWidget for the 15 content tabs and a custom CollapsibleSidebar widget. A single QTimer with a 1,000 ms interval drives all data refreshes. The refresh cycle calls ProcReader::readAll() which opens and reads all five primary /proc files sequentially; the total read time observed in testing was consistently under 5 ms even with the maximum 2,048 connection entries.

Component	Technology	Version	Purpose
Kernel Module	C (GNU C)	GCC 11.4	Netfilter hooks, proc interface, DNS parser
Component	**Technology**	**Version**	**Purpose**
Build System (Kerne	l)Makefile + Kbuild	Kernel 6.8	Kernel module compilation
GUI	Qt6 / C++	Qt 6.4.2	Cross-platform graphical interface
GUI Build	CMake	3.22+	GUI compilation with Qt6 find_package
Route Daemon	Python 3	3.10.12	Traceroute orchestration and GeoIP enrichment
GeoIP	MaxMind GeoLite2	Jan 2025	City, country, ASN, lat/lon lookup
History DB	SQLite3	3.37	Per-process bandwidth history
Packet Trace	traceroute	2.1.0	TCP traceroute -T -p 443
OS	Ubuntu 22.04 LTS	22.04.3	Target deployment platform
Kernel Target	Linux	6.8.0-106	Netfilter, RCU, workqueue APIs

Table 3.3 — Technology stack

The data flow through the KTA system follows a well-defined pipeline from packet arrival to GUI display. The following table describes each stage in the pipeline:

Stage	Component	Input	Output	Latency
1. Packet arrival	Kernel netfilter	Raw skb from NIC	skb at hook	<1 µs
2. DNS check	netfilter_hook.c	UDP/53 skb	IP domain in dns_map	<2 µs
3. Packet parse	packet_parser.c	skb at hook	flow_key, protocol info	1–3 µs
4. PID resolution	resolver.c (async)	flow_key	PID + exe path	0–100 ms
5. Stats update	stats.c	flow_key + PID	connection entry, anoma	l<y1flµagss
6. Proc export	proc_interface.c	connection table	/proc files	on read
7. Route request	proc_interface.c	new dest IP	PENDING in route_store	<1 µs
8. Route daemon	ta_route_daemon.py	PENDING IP	traceroute + GeoIP data	10–60 s
9. Route write	proc_interface.c write	DEST/HOP lines	hop data in route_store	<1 µs
10. GUI refresh	ProcReader.cpp	/proc files	data model update	1 s interval
11. GUI render	TrafficModel, widgets	data models	screen pixels	<5 ms
12. Netlink event	netlink_comm.c	connection change	multicast message	<1 µs
13. GUI netlink	GUI NetlinkListener	netlink message	immediate GUI	<10 ms

			update	

Table 3.4 — KTA data flow pipeline stages

The seven /proc files exposed by KTA form the primary interface between the kernel module and the GUI. Each file is implemented as a seq_file backed virtual file in proc_interface.c. All files use pipe-delimited (|) columns for easy parsing with str.split('|') in Python or

QString::split('|') in Qt C++. The following sections describe each file in detail.

/proc/traffic_analyzer — Main Connection Table

This is the primary output file, containing one row per tracked TCP/UDP connection. It is read by ProcReader::readConnections() every 1 second. The 22 columns are:

Column	Type	Description
PID	Integer	Process ID of the attributed process. 0 if unresolved.
UID	Integer	User ID of the process owning the socket.
PROCESS	String	Process name (comm field, up to 15 chars).
RESOLVED	Boolean (0/1)	1 if PID attribution is confirmed, 0 if pending.
STATE	String	TCP state: SYN_SENT, ESTABLISHED, FIN_WAIT, CLOSED, or UDP.
DNS	Boolean (0/1)	1 if the destination domain was resolved from DNS cache.
PROTO	String	Protocol: TCP or UDP.
SRC_IP	IPv4 dotted	Source IP address.
DEST_IP	IPv4 dotted	Destination IP address.
SRC_PORT	Integer	Source port number.
DEST_PORT	Integer	Destination port number.
DOMAIN	String	Domain name from DNS cache, or empty string.
OUT_BYTES	Long	Total outgoing bytes since connection creation.
IN_BYTES	Long	Total incoming bytes since connection creation.
OUT_PKTS	Long	Total outgoing packets.
IN_PKTS	Long	Total incoming packets.
RATE_OUT_BPS	Long	Current 1-second outgoing rate in bytes/sec.
RATE_IN_BPS	Long	Current 1-second incoming rate in bytes/sec.
FIRST_SEEN	Long (Unix)	ktime_get_real_seconds() at connection creation.
LAST_SEEN	Long (Unix)	ktime_get_real_seconds() at last packet.
DURATION	Long (secs)	LAST_SEEN - FIRST_SEEN.
CLOSED_AT	Long (Unix)	ktime_get_real_seconds() when CLOSED state was entered, or 0.

Table 3.5 — /proc/traffic_analyzer column definitions

/proc/traffic_analyzer_procs — Process Aggregation Table

This file aggregates per-connection data up to the process level. One row per process with active connections. Columns include PID, UID, PROCESS, EXE (full path), TCP_CONNS, UDP_CONNS, TOTAL_CONNS, SYN_PENDING, OUT_BYTES, IN_BYTES, OUT_PKTS, IN_PKTS, RATE_OUT_BPS, RATE_IN_BPS, TCP_PCT, UDP_PCT, ANOMALY flag, and
TOP1–TOP5 (top destination IPs by byte volume for this process). This file is read by

ProcReader::readProcesses().

Other /proc Files

File	Description
/proc/traffic_analyzer_dns	DNS flow entries showing per-process DNS query activity.
/proc/traffic_analyzer_anomalies	11 columns: PID, PROCESS, TYPE, FIRST_SEEN, LAST_SEEN, COUNT, TH
/proc/traffic_analyzer_dns_map	7 columns: DOMAIN|IP|TTL_REMAINING|QUERIED_BY_PID| QUERIED_BY
/proc/traffic_analyzer_routes	15 columns per hop: DEST_IP|DOMAIN|STATUS|TOTAL_HOPS|HOP_N| HO
/proc/	Read/write file. Reading returns PENDING IPs (atomically transitions

Table 3.6 — Additional /proc file descriptions

RESHOLD,

_COMM|FIRST

P_IP|HOST|RT

kti me _ge t_ns () for tim est am ps. The foll owi ng

KTA Phase 6 introduces a push-based netlink multicast channel that eliminates the need for the GUI to poll /proc files for every state change. The implementation uses NETLINK_USERSOCK with protocol number 2, which is an unreserved protocol slot available for user-defined kernel-userspace communication.

Four multicast groups are defined: group 1 (connections) receives CONN_NEW, CONN_UPDATE, and CONN_CLOSED messages; group 2 (anomalies) receives ANOMALY messages; group 3 (DNS) receives DNS_RESOLVED messages; and group 4 (routes) receives ROUTE_READY messages. The GUI subscribes to all four groups by calling setsockopt with NETLINK_ADD_MEMBERSHIP.

Message Type	Group	Struct	Key Fields
CONN_NEW	1 (connections)	ta_msg_conn	pid, uid, process, src_ip, dst_ip, src_port, dst_port, proto
CONN_UPDATE	1 (connections)	ta_msg_conn	Same as CONN_NEW plus state, rate_out_bps, rate_in_
CONN_CLOSED	1 (connections)	ta_msg_conn	Same as CONN_NEW plus closed_at timestamp
ANOMALY	2 (anomalies)	ta_msg_anomaly	pid, process, anomaly_type, value, threshold, timestamp
DNS_RESOLVED	3 (DNS)	ta_msg_dns	ip, domain, ttl, queried_by_pid, queried_by_comm
ROUTE_READY	4 (routes)	ta_msg_route	dest_ip, total_hops, hops[32] (inline)

Table 3.7 — Netlink message types and their fields

To prevent message flooding, the netlink emission in stats.c implements a throttle policy: a CONN_UPDATE message is sent only if (a) the rate has changed by more than 10% since the last message, or (b) 5 seconds have elapsed since the last forced update. State changes (SYN_SENT ESTABLISHED, ESTABLISHED FIN_WAIT, etc.) always trigger an immediate message regardless of the throttle. This policy ensures that the GUI receives timely updates for important events while avoiding a flood of minor rate variation messages.

All five anomaly detection algorithms operate on per-process velocity counters maintained in the stats.c velocity_tracker struct. The tracker is updated on every call to stats_update() and maintains a 1-second sliding window using

ING). Writing

, domain bps

pseudocode-style descriptions capture the key logic.

Algorithm 1: PORT_SCAN Detection

```
Each process maintains a unique_dstports bitmap (512 bits, one per port / 128).
On each outgoing TCP SYN packet: 1. If (ktime_now - window_start) > 1 second:
reset unique_dstports, window_start = ktime_now 2. Set bit (dst_port % 512) in
unique_dstports 3. port_count = popcount(unique_dstports) 4. If port_count >=
ANOMALY_PORT_SCAN_PORTS (15): set ANOMALY_PORT_SCAN flag
```

The bitmap approach avoids storing individual port numbers and allows constant-time unique port counting via popcount(). The 512-bit bitmap uses only 64 bytes, well within the per-process struct's allocation budget.

Algorithm 2: SYN_FLOOD Detection

```
Each process maintains syn_pending_count and total_tcp_count. 1. On TCP SYN
sent: increment syn_pending_count and total_tcp_count 2. On TCP SYN+ACK
received (matching PID): decrement syn_pending_count 3. On any state change to
ESTABLISHED: syn_pending_count = max(0, syn_pending_count - 1) 4. ratio =
syn_pending_count / max(total_tcp_count, 1) 5. If ratio >= ANOMALY_SYN_RATIO
(0.80) AND total_tcp_count >= 10: set ANOMALY_SYN_FLOOD flag
```

The minimum connection count guard (total_tcp_count >= 10) prevents false positives immediately after a process starts making its first few connections.

Algorithm 3: CONN_BURST Detection

```
Each process maintains new_conns_this_second counter. 1. If (ktime_now -
window_start) > 1 second: reset new_conns_this_second, window_start = now 2. On
each new connection attributed to PID: increment new_conns_this_second 3. If
new_conns_this_second >= ANOMALY_CONN_BURST (20): set ANOMALY_CONN_BURST flag
```

Algorithms 4 & 5: HIGH_CONNS and HIGH_BW Detection

```
HIGH_CONNS: if process.total_conns >= ANOMALY_HIGH_CONNS (200): set flag
HIGH_BW: if (rate_out_bps + rate_in_bps) >= ANOMALY_HIGH_BW (10485760): set
flag (10485760 = 10 * 1024 * 1024 bytes/sec = 10 MB/s)
```

Whitelisting is implemented via is_system_process() which checks the process comm field against a static list: {traceroute, ping, nmap, curl, wget, iperf, iperf3}. Whitelisted processes have all anomaly flags suppressed. This prevents the route daemon's traceroute invocations from triggering persistent SYN_FLOOD and PORT_SCAN false positives.

KTA's DNS pipeline operates in two phases: passive response interception and active connection enrichment. Phase 1 runs in the NF_INET_LOCAL_IN hook when a UDP packet arrives from source port 53 (a DNS response). Phase 2 runs in parse_packet() when a new connection entry is created.

In Phase 1, dns_parser.c receives the UDP payload (skb_header_pointer to the DNS message) and parses it according to RFC 1035 wire format. The parser reads the Question and Answer sections, following label pointers (compression) with a 10-jump loop guard to prevent pointer loops from causing infinite loops. For each Answer record with class IN (0x0001) and type A (0x0001), the parser extracts the 4-byte RDATA as an IPv4 address and the record's TTL. Each IP domain mapping is inserted into dns_map using spin_lock_bh-protected hash table operations. Up to 64 answer records are processed per response.

In Phase 2, when parse_packet() creates a new traffic_entry for a connection, it calls dns_map_lookup(dst_ip) to check if the destination IP was recently resolved. If a match is found, the domain field of the traffic_entry is populated immediately and the DNS flag is set to
1. Because DNS responses always precede the TCP SYN that initiates the connection (a TCP connection cannot be opened to a host whose IP is unknown), this lookup succeeds for the vast majority of connections initiated by well-behaved applications.

DNS TTL tracking ensures that stale entries are not shown indefinitely. The dns_map entry stores the TTL value from the DNS response and decrements it on each proc_list_rebuild() (called every second from the stats timer). Entries with TTL=0 are removed and their domains are no longer available for new connection enrichment. Existing connection entries retain their populated domain field even after the dns_map entry expires.

As a kernel module that processes every network packet on the system, KTA requires root privileges for loading and operates in the highest trust domain. Several security considerations informed the implementation:

- **No packet modification:** All netfilter hooks return NF_ACCEPT unconditionally. KTA never modifies, delays, or drops packets. This ensures that KTA cannot be used to intercept or tamper with data in transit.
- **Proc interface access control:** The /proc files created by KTA use the default proc permissions (0444 for read-only files, 0644 for routes_pending which supports write). Access is controlled by the kernel's standard proc file permission model; root or a process with CAP_NET_ADMIN is required to load the module.
- **Softirq-safe synchronisation:** All data structures accessed from both softirq context (the netfilter hooks) and process context (workqueue resolvers, proc reads) are protected with spin_lock_bh(). This is the correct Linux kernel primitive for structures shared between bottom-half (softirq) and process context. Using spin_lock() rather than spin_lock_bh() in softirq context would allow nested softirq preemption and potential deadlock.
- **Memory safety:** All dynamic allocations use GFP_ATOMIC in softirq context. Allocation failures are handled gracefully: if kmalloc returns NULL, the packet is still accepted (NF_ACCEPT) and the connection entry is not created, resulting in a missed attribution rather than a kernel panic.
- **Bounded data structures:** The connection table (2,048 entries), DNS map (configurable, default 4,096 entries), and route store are all bounded. This prevents unbounded memory growth under adversarial traffic conditions such as a SYN flood that would otherwise create millions of connection entries.

This chapter presented the five research gaps motivating KTA, followed by a detailed description of the three-layer architecture and all major subsystems. The proc file interface contract was documented column by column. The netlink protocol design including message types, multicast groups, and throttle logic was explained. Pseudocode-style descriptions of all five anomaly detection algorithms were provided, with reference to the specific threshold constants used in the

implementation. The DNS pipeline's two-phase operation was described in detail. Security considerations covering privilege model, memory safety, and synchronisation

were discussed. Chapter 4 now presents experimental results that validate the system's performance against the seven objectives defined in Chapter 1.

CHAPTER 4 Results and Discussions

This chapter presents the experimental results obtained from deploying KTA on a Ubuntu 22.04 system with kernel 6.8.0-106-generic. All tests were conducted on a machine connected through a 4G mobile hotspot to simulate the most restrictive network environment targeted by the project. Results are compared against the seven objectives defined in Chapter 1.

Parameter	Value
CPU	Intel Core i5-8250U, 4 cores / 8 threads @ 3.4 GHz boost
RAM	8 GB DDR4-2400, dual channel
Storage	256 GB NVMe SSD
OS	Ubuntu 22.04.3 LTS (Jammy Jellyfish)
Kernel	6.8.0-106-generic (HWE stack)
Network	4G LTE hotspot (~30 Mbps down, ~10 Mbps up)
Python	3.10.12
Qt	6.4.2
GeoLite2 DB date	January 2025
traceroute	2.1.0
Test duration	72 hours continuous monitoring
Traffic load (synthetic)	100 Mbps peak (iperf3 loopback for overhead tests)

Table 4.1 — Experimental setup

Over the 72-hour test period, KTA tracked a total of 84,312 unique TCP/UDP connections from 47 unique processes. PID attribution accuracy was measured by comparing KTA's attributed PID against ground-truth PID from /proc/net/tcp6 for a stratified sample of 10,000 connections across short-lived (< 500 ms) and long-lived (>= 500 ms) flows.

Metric	Value
Total connections tracked	84,312
Unique processes observed	47
PID attribution accuracy (all)	96.8%
PID attribution accuracy (TCP 500 ms)	99.1%
Connections via sock_cache hit	71.2%
Connections via inode_cache hit	18.4%

Connections via flow_cache hit	6.1%
Metric	**Value**
Connections via full resolver	1.1%
Unresolved (primarily kernel threads + short UDP	)3.2%
Average stats_update() latency (ktime)	< 2 µs
Max traffic_list size (peak load)	1,847 entries
Connection table evictions (72 h)	0 (peak never exceeded 2,048)

Table 4.2 — Connection tracking statistics (72-hour run)

The sock_cache hit rate of 71.2% confirms that the cache hierarchy is highly effective at avoiding expensive for_each_process() scans. The remaining 3.2% unresolved rate is primarily attributable to: (a) kernel threads that generate network traffic without a userspace process counterpart (approximately 1.8%); (b) short-lived UDP flows that closed before the workqueue resolver completed (approximately 1.4%). These cases are structurally unavoidable given the softirq context constraint.

Anomaly detection was evaluated by deliberately triggering each anomaly type with specific tools and measuring detection latency (time from first offending packet to ANOMALY flag being set in the kernel data structure) and false positive rate (anomaly flags set during normal operation without deliberate triggering, measured over a 24-hour baseline period).

Anomaly Type	Threshold	Trigger Tool	Detection Latency	False Positive Rate
PORT_SCAN	15 unique dst ports/sec	nmap -sS	1.0 – 1.5 s	0.8%
SYN_FLOOD	SYN ratio 80%	hping3 --flood	1.0 – 1.2 s	0.3%
CONN_BURST	20 new conns/sec	Custom Python script	1.0 – 1.1 s	1.2%
HIGH_CONNS	200 total connections	Apache Bench (ab)	< 1.0 s	0.1%
HIGH_BW	10 MB/s combined	iperf3	1.0 – 2.0 s	0.5%

Table 4.3 — Anomaly detection evaluation results

CONN_BURST had the highest false positive rate (1.2%) due to web browsers opening multiple parallel connections on page load. Modern browsers (Chrome, Firefox) typically open 6–15 parallel TCP connections per domain to parallelize resource loading. While this is well below the 20/second threshold, certain JavaScript-heavy pages that trigger multiple domain lookups simultaneously can briefly exceed it. This is addressed in future work by implementing browser process whitelisting or a configurable per-process threshold.

Route tracing was tested against 200 unique destination IPs observed during the monitoring period. All tests used TCP mode (-T -p 443) through the 4G hotspot.

Metric	Value
Total destination IPs traced	200
Metric	**Value**
Successful (1 hop returned)	192 (96%)
Failed (symmetric NAT, no hops)	8 (4%)
Average hop count	12.4 hops
Average traceroute time per destination	28 seconds
GeoIP city resolution rate (public IPs)	87%
ASN resolution rate (public IPs)	98%
Simulated BGP changes detected	2 of 2 (100%)

Table 4.4 — Route tracing and geo-enrichment results

The 4% failure rate corresponds to destination IPs behind symmetric NAT (common for some gaming and P2P services) where no intermediate router sends ICMP Time Exceeded messages. The 87% city resolution rate for public IPs reflects the inherent limitation of MaxMind GeoLite2: some IPs (particularly those allocated to cloud providers, CDNs, and anycast services) are geolocated only at the country or continent level, with no specific city data.

The Qt6 GUI was evaluated for CPU and memory usage during continuous operation over 8 hours with approximately 150 active connections, representing a typical developer workstation workload.

Metric	Value
GUI CPU usage (average)	2.1% on one core
GUI CPU usage (range)	1.2% – 3.5%
GUI memory usage (RSS)	48 MB stable over 8 hours
SQLite history DB growth	~12 KB / hour
Kernel module CPU overhead	< 0.05% at 100 Mbps
SparklineDelegate render time	< 0.3% CPU
Route map repaint interval	30 ms (smooth animation)
iptables rule apply latency	< 50 ms
tc throttle rule apply latency	< 80 ms

JSON export (150 conns, 200 routes)	< 200 ms
PDF export (full report)	< 1.5 seconds
All 15 tabs functional	Yes

Table 4.5 — GUI performance metrics

DNS resolution enrichment was evaluated by measuring what percentage of new TCP connections had their domain field populated at connection creation time (immediate enrichment) versus after a subsequent DNS response (retroactive enrichment) versus never (unresolved).

Metric	Value
Total unique connections (TCP + UDP)	84,312
Connections with domain at creation time	79,253 (94.0%)
Connections enriched retroactively	1,847 (2.2%)
Connections with no domain (no DNS observed)	3,212 (3.8%)
DNS responses parsed (72 h)	312,847
DNS A records extracted	1,124,392
Average TTL in dns_map at time of connection	187 seconds
DNS map peak size	3,847 entries
DNS pointer decompression errors	0
DNS parse failures (malformed responses)	3 (< 0.001%)

Table 4.6 — DNS resolution performance statistics

The 94.0% immediate enrichment rate confirms that the in-kernel DNS interception approach is highly effective. The 3.8% unresolved rate corresponds to connections to hardcoded IP addresses (common in mobile apps and IoT firmware), connections using encrypted DNS (DNS-over-HTTPS, which uses TCP port 443 and appears as HTTPS traffic rather than DNS), and connections where the DNS response arrived after the connection was already established.

Kernel module memory usage was measured using /proc/slabinfo and direct ksize() calls on the allocated structures. The following table summarises the memory footprint of each major kernel data structure.

Data Structure	Max Entries	Bytes per Entry	Max Total
traffic_entry (connection table)	2,048	~320 bytes	655 KB
proc_entry (process table)	512	~280 bytes	140 KB
dns_map entry	4,096	~128 bytes	512 KB
route_store hop data	2,048 (32 hops each)	~64 bytes/hop	4 MB
sock_cache	4,096	~32 bytes	128 KB
inode_cache	2,048	~32 bytes	64 KB
flow_cache	2,048	~48 bytes	96 KB
Total (approximate maximum)	—	—	~5.6 MB

Table 4.7 — Kernel module memory footprint at maximum capacity

The total maximum kernel memory footprint of approximately 5.6 MB is negligible on any system with 1 GB or more of RAM. In practice, the observed peak memory usage during the 72-hour test was approximately 2.1 MB, as the connection table never reached its 2,048-entry limit. The kernel module itself is approximately 180 KB compiled size (modinfo KTA.ko).

A direct performance comparison was conducted between KTA and three representative existing tools: tcpdump (packet capture), nethogs (process bandwidth monitoring), and ss
-tulnp (connection snapshot). Each tool was run simultaneously under identical conditions for 30 minutes.

Metric	tcpdump	nethogs	ss -tulnp	KTA
CPU overhead	3–8% (100% pkt	r0at5e-) 1%	< 0.01% (snapsh	o<t)0.05%
Memory (process)	~80 MB (PCAP	bu~f8erM) B	Negligible	Kernel: 2.1 MB GUI: 48 MB
Attribution accuracy	None	~85%*	~90%*	96.8%
Short-lived conn coverag	1e00% (PCAP)	~60%	~30%	~97%
DNS enrichment	Manual post-proc	eNsosne	None	94% automatic
Geo routing	None	None	None	96% of IPs
Anomaly detection	Manual	None	None	5 types, < 2 s
Firewall integration	None	None	None	One-click iptables
Real-time GUI	No	Ncurses	No	Qt6, 15 tabs

*Table 4.8 — Quantitative comparison with existing tools * nethogs and ss attribution accuracy estimated from published benchmarks*

This case study demonstrates KTA's PORT_SCAN anomaly detection in a realistic scenario. An attacker running nmap on the monitored machine (simulating a compromised host or a red team exercise) initiates a SYN scan against an internal network range.

Scenario Setup

The attacker runs: nmap -sS 192.168.1.0/24 -p 1-1000 from the monitored machine. This generates approximately 254 × 1000 = 254,000 SYN packets. nmap's default timing template (T3) sends approximately 500 SYN packets per second. At this rate, unique destination ports exceed the 15/second threshold within the first second of scanning.

Step-by-Step Detection Timeline

Time (s)	Event	KTA Action
T+0.000	nmap starts, first SYN sent to 192.168.1.1	:1sock_cache miss workqueue resolver queued

T+0.020	Resolver completes: PID=12847, comm=n	msoaqpk_cache populated, proc_entry created for nmap
T+0.100	50+ SYN packets sent to unique ports	port_count in velocity_tracker = 50

Time (s)	Event	KTA Action
T+1.000	End of first 1-second window	port_count = 500 >> threshold (15), ANOMALY_PORT_SCA
T+1.001	stats.c anomaly check fires	Netlink ANOMALY message emitted to GUI
T+1.010	GUI netlink listener receives message	AlertPopup slides in: "PORT_SCAN: nmap (PID 12847)"
T+1.010	TrayIcon anomaly badge incremented	Badge shows "1" in system tray
T+1.500	Anomaly visible in Anomalies tab	Row: nmap | PORT_SCAN | 500 ports/s | threshold 15
T+2.000	Second 1-second window counted	Anomaly flag remains set, count increments

Table 4.9 — Port scan detection timeline

The analyst can respond immediately by clicking the Block button on the nmap process in the Processes tab, which triggers FirewallManager::blockProcess(12847) and inserts an iptables OUTPUT -m owner --pid-owner 12847 -j DROP rule, halting the scan within 50 ms. Alternatively, the analyst can throttle nmap's bandwidth to 1 KB/s using the BandwidthThrottler, effectively stopping the scan without killing the process.

This case study demonstrates that the combination of kernel-level rate tracking, push-based netlink notification, and integrated firewall control enables a response time from first offending packet to active countermeasure of under 2 seconds — compared to several minutes for a manual workflow using separate tools.

Scenario Setup

A malicious process (simulated by a Python script) opens a TCP connection to an external server and begins uploading data at 15 MB/s — exceeding the HIGH_BW threshold of 10 MB/s. The process disguises itself with the name "systemd-update" to avoid casual inspection.

Detection and Response

Time (s)	Event	KTA Response
T+0	Malicious process starts, TCP connect	ioCnonensteacbtiliosnh edntry created: PID=9934, state=ESTABLISHED
T+0.5	Upload rate reaches 15 MB/s	rate_out_bps = 15,728,640 (> 10,485,760 threshold)
T+1.0	HIGH_BW anomaly flag set	Netlink ANOMALY sent: "HIGH_BW: systemd-update (PID 993
T+1.01	AlertPopup shown	"HIGH_BW: systemd-update uploading 15 MB/s to 185.220.101
T+1.01	RouteMap animates new arc	Bezier arc from Bhopal to destination country visualised
T+3	Analyst inspects connection	Domain: (none — hardcoded IP), Dest country: DE (Germany)
T+4	Analyst clicks "Block"	iptables DROP rule added, upload stops immediately
T+60	Historical record in HistoryDB	Exfiltration event persisted to SQLite for forensic review

Table 4.10 — High bandwidth exfiltration detection timeline

A key observation from this case study is that the lack of a DNS domain for the destination IP (the process connected directly to an IP address without prior DNS

resolution) is itself a suspicious indicator. KTA's DNS Map tab would show no entry for 185.220.101.x, while the

N set

4)"

.x"

Route Map would show the hop path traversing through foreign ASNs. The combination of HIGH_BW anomaly, no DNS enrichment, and foreign routing constitutes a strong exfiltration signal.

Scenario Setup

BGP route changes are simulated by modifying /etc/hosts to redirect a known domain (news.example.com) from its normal IP (203.0.113.1, ASN 12345, India) to a different IP (198.51.100.1, ASN 67890, Russia). This simulates the effect of a BGP hijacking event as observed from the endpoint perspective.

Detection

Time (s)	Event	KTA BGP Monitor Response
Day 1–7	Normal routing: news.example.com	ABSasNe1lin2e34e5s,taINblished in BgpMonitor
Day 8, T+0	/etc/hosts modified to redirect domain	Next DNS response returns new IP
Day 8, T+30	New route trace completed for 198.51.	1A0S0.N1 67890 (RU) detected — differs from baseline
Day 8, T+31	BGP Monitor comparison fires	Alert: "BGP CHANGE: news.example.com ASN 12345 678
Day 8, T+31	BGPMonitor tab shows alert row	Old ASN: 12345 (IN), New ASN: 67890 (RU), timestamp log

Table 4.11 — BGP route change detection timeline

This case study illustrates the value of maintaining per-domain routing baselines. Without KTA's BGP Monitor, the route change would be invisible to the user unless they periodically ran manual traceroutes. The automated baseline comparison provides a practical first line of defence against routing anomalies without requiring integration with internet-scale BGP monitoring infrastructure.

Several limitations were observed during the 72-hour test that were not anticipated in the design phase:

- **CONN_BURST false positives from package managers:** apt and snap were observed opening 25–40 parallel connections during package update checks, briefly exceeding the CONN_BURST threshold. These were addressed by adding apt and snap to the whitelist for CONN_BURST only (not for PORT_SCAN or SYN_FLOOD).
- **DNS-over-HTTPS blind spot:** Firefox in its default configuration uses DNS-over-HTTPS through Cloudflare (1.1.1.1:443). These DNS queries are indistinguishable from regular HTTPS traffic at the packet level. Approximately 12% of Firefox connections had no DNS enrichment for this reason.
- **traceroute timing variability:** On the 4G hotspot, traceroute completion times ranged from 8 to 94 seconds depending on network congestion. The 2-second polling interval in the route daemon means that during peak traffic periods, the route queue grew to 15–20 pending IPs, delaying new route results by up to 3 minutes.
- **Kernel module compilation sensitivity:** The module was compiled against the exact running kernel version (6.8.0-106-generic). Installation after a kernel

90, IN RU"

ged

6.8.0-107) requires recompilation. DKMS (Dynamic Kernel Module Support) integration would automate recompilation but was not implemented in the current version.

The experimental results validate all seven objectives defined in Chapter 1. Objective 1 (kernel-level packet interception) is confirmed by the 0 dropped packets across 84,312 tracked connections. Objective 2 (per-process attribution) is confirmed by 96.8% PID attribution accuracy with < 0.05% CPU overhead. Objective 3 (in-kernel DNS parsing) is confirmed by 94.0% immediate domain enrichment. Objective 4 (route daemon) is confirmed by 96% successful route tracing through 4G NAT. Objective 5 (anomaly detection) is confirmed by sub-2-second detection latency across all five anomaly types. Objective 6 (GUI) is confirmed by stable 2.1% CPU usage across all 15 tabs. Objective 7 (integrated remediation) is confirmed by
< 50 ms iptables rule application and < 80 ms tc throttle application.

The most significant design trade-off revealed by testing is the latency introduced by the 1-second rate window: anomaly detection latency is fundamentally bounded by this window. A sub-second window is theoretically possible but would increase the false positive rate for bursty but legitimate traffic (e.g., browsers on page load). The 1-second window represents a practical balance between detection speed and false positive rate for the targeted use case.

CHAPTER 5 Conclusions and Future Work

This project designed, implemented, and evaluated the Kernel Traffic Analyzer (KTA) v7.0 — a production-grade Linux network observability system. The following conclusions are drawn from the experimental results presented in Chapter 4.

Conclusion 1 — Kernel-level monitoring is superior to userspace polling: The 96.8% PID attribution accuracy (99.1% for TCP connections > 500 ms) and zero dropped connections over 84,312 tracked flows demonstrate that netfilter-based capture provides fundamentally more complete visibility than /proc polling. The 3.2% unresolved rate is inherent to the PID attribution problem in softirq context rather than a limitation of the capture mechanism.

Conclusion 2 — In-kernel DNS parsing enables near-instant enrichment: The 94.0% immediate domain enrichment rate validates the design decision to parse DNS responses in the netfilter hook before connection tracking runs. This approach requires only 312,847 DNS response parsings over 72 hours (averaging 1.2 parsings per second), well within the performance budget of a softirq hook.

Conclusion 3 — TCP traceroute on port 443 is effective for 4G environments: The 96% successful route tracing rate confirms that traceroute -T -p 443 is the correct choice for environments with carrier-grade NAT. The 4% failure rate corresponds to symmetric NAT scenarios that are architecturally opaque to any traceroute mechanism.

Conclusion 4 — Threshold-based anomaly detection achieves practical accuracy: False positive rates below 1.5% for all five anomaly types, combined with sub-2-second detection latency, meet the design objectives. The CONN_BURST false positive rate (1.2%) for browser traffic is the primary area for improvement, addressable by per-browser threshold tuning or machine learning-based profiling.

Conclusion 5 — The three-layer architecture achieves excellent resource efficiency: The kernel module overhead of less than 0.05% CPU at 100 Mbps and the GUI's 48 MB stable RSS with 2.1% average CPU usage demonstrate that a comprehensive monitoring system need not impose significant performance penalties. This makes KTA suitable for deployment on developer workstations without perceptible impact on the user's primary workflow.

Conclusion 6 — Integrated remediation significantly reduces response time: The case studies in Sections 4.9–4.11 demonstrate that the combination of real-time anomaly alerts, route visualisation, and one-click iptables/tc control enables a response time from detection to countermeasure of under 5 seconds, compared to several minutes for a manual workflow using separate tools.

The following eight directions are identified for future enhancement based on limitations observed during testing and feedback from the project evaluation:

- **Full IPv6 support:** Extend flow_key and traffic_entry to store 128-bit IPv6 addresses using two uint64_t fields. Update ProcReader.cpp to parse IPv6 notation. Update RouteMapWidget to handle IPv6 geolocation. Expected effort: 3–5 days.

- **Machine learning anomaly detection:** Replace fixed-threshold detectors with an online Isolation Forest model trained on per-process behavioural baselines. This would reduce CONN_BURST false positives from browser traffic while improving detection of novel slow-rate attacks. The model could be implemented in the Python route daemon process to avoid kernel complexity.
- **Generic Netlink family:** Replace NETLINK_USERSOCK with a properly registered Generic Netlink family using genl_register_family(). This provides better multi-process subscription management and is forward-compatible with future kernel netlink policy enforcement.
- **Container-aware monitoring:** Add network namespace awareness using struct net* from skb->dev->nd_net to distinguish Docker/Podman containers. Map network namespace IDs to container names via the Docker API. This would enable per-container traffic attribution.
- **eBPF co-implementation:** Provide an eBPF alternative implementation of the packet capture and PID attribution pipeline for environments where loading unsigned kernel modules is prohibited (Secure Boot, locked-down kernels). The /proc interface contract remains identical.
- **Threat intelligence integration:** Replace the heuristic ThreatIntel scorer with live feeds from AbuseIPDB or Shodan's InternetDB for real-time reputation-based scoring of destination IPs.
- **Distributed monitoring:** Add a lightweight protobuf-based aggregation protocol allowing multiple KTA instances to report to a central server. This enables fleet-wide anomaly correlation.
- **Web dashboard:** Export the live data stream via WebSocket from a small embedded HTTP server (libmicrohttpd) to enable browser-based monitoring without requiring Qt6 installation.

The development of KTA over seven major iterations yielded several valuable engineering lessons that are worth documenting for future kernel module developers.

The most significant lesson was the importance of understanding kernel context discipline. The bug class of "spin_lock() called from softirq context" (which should always be spin_lock_bh()) caused several hours of debugging in early versions when the system would intermittently deadlock under high packet rates. The fix was straightforward once diagnosed, but the root cause was subtle: softirq preemption can occur on any CPU at any time, and if a softirq attempts to acquire a spinlock that is already held by process context on the same CPU (with the softirq preempting after the lock was taken but before it was released), a deadlock results.

The second major lesson was the removal of get_mm_exe_file() in kernel 6.7. This API had been available since the earliest days of the kernel and was used in dozens of tutorials and example kernel modules. Its removal without deprecation warning (it was simply removed in a commit) broke any module that used it. The replacement pattern (rcu_dereference(mm->exe_file) under RCU read lock) is cleaner and correct, but discovering it required reading the kernel commit history.

The third lesson was the importance of the SparklineDelegate vtable constraint.

Adding Q_OBJECT to a QStyledItemDelegate subclass used as a delegate for a QTableView column caused a cryptic vtable linker error at compile time. The fix (removing Q_OBJECT and implementing the delegate without signals/slots) is described in Qt developer forums but not in

the official documentation. This cost approximately 4 hours of debugging.

Finally, the make_canonical() function that normalises the 4-tuple by placing the lower IP in the src field was introduced in version 3 to deduplicate bidirectional flows. However, the compensating change in ProcReader.cpp that swaps src/dst when reading the proc file back was not made until version 4, causing a period where the GUI showed internal IPs as destinations and external IPs as sources for incoming connections.

KTA was developed over approximately 14 months across seven major versions. The following table summarises the development phases.

Phase / Version	Duration	Key Deliverables
v1.0 — Proof of Concep	tWeeks 1–3	Basic netfilter hook, /proc/traffic_analyzer with 8 columns, no PID attributio
v2.0 — PID Attribution	Weeks 4–7	sock_cache, inode_cache, flow_cache, resolver workqueue. Attribution ac
v3.0 — DNS Integration	Weeks 8–11	dns_parser.c RFC 1035 implementation, dns_map, DNS enrichment. 22-c
v4.0 — Route Daemon	Weeks 12–15	ta_route_daemon.py, MaxMind integration, route_store, /proc/traffic_analy
v5.0 — Qt6 GUI	Weeks 16–22	Full Qt6 GUI with 15 tabs, TrafficModel, ProcModel, RouteMapWidget, His
v6.0 — Anomaly Detecti	oWn eeks 23–27	5 anomaly types, velocity_tracker, AlertPopup, TrayIcon, FirewallManager
v7.0 — Netlink + Polish	Weeks 28–34	Phase 6 netlink multicast, DnsLeakTab, BgpMonitor, PDF/JSON/CSV exp
Testing & Documen	tWateioenks 35–56	72-hour test run, case studies, thesis writing, GitHub cleanup

n

curacy ~72%

olumn proc

for

zer_routes

toryDB

,

BandwidthTh

r ort, dark

theme

Table 5.1 — KTA development timeline

Objective	Status	Evidence
1. Kernel-level TCP/UDP interception with	oMutept acket dro	p8s4,312 connections tracked; 0 dropped packets
2. Per-process attribution via 4-level cache	Met	96.8% accuracy; 71.2% sock_cache hit rate
3. In-kernel DNS parsing (RFC 1035)	Met	94.0% immediate enrichment; 312,847 responses parsed
4. Route daemon with MaxMind geo-enric	hMmetnt	96% of IPs traced; 98% ASN resolution
5. Five anomaly detectors with < 2 s laten	cMy et	All 5 types detected; false positives < 1.5%
6. Qt6 GUI with 15 tabs, 1 s refresh	Met	2.1% CPU; 48 MB RSS; all 15 tabs functional
7. Integrated iptables/tc remediation	Met	< 50 ms block; < 80 ms throttle application

Table 5.2 — Objectives achievement summary

KTA v7.0 represents the successful completion of a complex, multi-layer systems project that spans kernel programming, network engineering, geographic data

processing, and modern GUI development. All seven project objectives have been met. The system has been validated through a 72-hour continuous test with quantitative measurements across all major subsystems.

The project has been open-sourced on GitHub (https://github.com/Anilkumar64/kernal-traffic-analyzer) to enable other developers and security researchers to use, study, and extend the codebase. The author hopes that KTA will serve as both a practical tool for the Linux security community and an educational resource for developers learning kernel programming.

The author acknowledges the guidance of Prof. [Supervisor Name] at [Your University Name], whose direction during the project definition phase was instrumental in scoping the system to a feasible yet academically significant set of objectives. The Linux kernel developer community's extensive documentation, kernel.org source code, and the netfilter mailing list archives were invaluable references throughout the implementation.

CHAPTER 6 Implementation Details

This chapter provides detailed implementation notes for the most technically significant subsystems in KTA, with reference to specific source files, data structures, and key design decisions.

The packet_parser.c file implements a 10-step pipeline that processes every TCP and UDP packet seen by the netfilter hooks. Each step is designed to fail gracefully, returning NF_ACCEPT to ensure that any processing error does not affect packet delivery.

Step	Function / Operation	Key Detail
1	IP version check	Read skb->data[0] >> 4 for IP version. Use this rather than skb->protocol, which may
2	Header pointer validatio	nskb_header_pointer() with sizeof(struct iphdr) to safely read IP header without assum
3	Protocol filter	Only process IPPROTO_TCP and IPPROTO_UDP; all others return immediately.
4	Port extraction	skb_header_pointer() for transport header; extract src/dst port from TCP or UDP hea
5	Flow key construction	Populate flow_key with {src_ip, dst_ip, src_port, dst_port, proto}. Call make_canonica
6	Existing entry lookup	Locked search of traffic_list for matching flow_key. If found, update stats and return.
7	Socket lookup	skb->sk provides struct sock* for outgoing packets. sk_to_pid() consults sock_cache;
8	PID waterfall	sock_cache inode_cache flow_cache async resolver. Each level queues wo
9	New entry creation	kzalloc(GFP_ATOMIC) for new traffic_entry. Populate all fields. Atomic

Table 6.1 — packet_parser.c 10-step processing pipeline

A critical implementation detail is the use of skb_header_pointer() rather than direct pointer arithmetic on skb->data. The kernel socket buffer (skb) may be non-linear (fragmented across multiple memory pages, common for large packets assembled via GRO). skb_header_pointer() handles non-linear skbs transparently, providing a pointer to a linear copy in a provided buffer when necessary. Using direct pointer arithmetic on fragmented skbs would cause incorrect reads and potential kernel memory corruption.

The TCP FSM in stats.c implements a simplified version of the RFC 793 state machine with five states relevant to per-connection monitoring: UNKNOWN, SYN_SENT, ESTABLISHED, FIN_WAIT, and CLOSED. The simplification omits states that are not observable at the packet level from a single endpoint (TIME_WAIT, CLOSE_WAIT, etc.).

Current State	Observed Packet	New State	Action
UNKNOWN	TCP SYN (outgoing)	SYN_SENT	Increment syn_pending_count
UNKNOWN	TCP SYN (incoming)	SYN_SENT	Record as passive open
SYN_SENT	TCP SYN+ACK	ESTABLISHED	Decrement syn_pending_count; emit CONN_NEW netlin

Current State	Observed Packet	New State	Action
ESTABLISHED	TCP data packet	ESTABLISHED	Update bytes/packets/rate
ESTABLISHED	TCP FIN	FIN_WAIT	Note fin_seen timestamp
FIN_WAIT	TCP FIN+ACK	CLOSED	Set closed_at; emit CONN_CLOSED netlink
Any	TCP RST	CLOSED	Immediate CLOSED; set closed_at; emit CONN_CLOSE
CLOSED	(any)	CLOSED (evictio	nEpnetmydrientga)ined for 30 s then evicted

D

Table 6.2 — TCP connection state machine transitions

The dns_parser.c implementation follows RFC 1035 Section 4 wire format precisely. The DNS message structure consists of a 12-byte fixed header followed by Question and Answer sections. The parser skips the Question section (counting questions from the QDCOUNT field) and processes the Answer section (ANCOUNT records) for IN-class A records.

Label encoding in RFC 1035 uses a length-byte followed by label octets, with a zero byte terminating the name. Pointer compression (RFC 1035 Section 4.1.4) allows a 2-byte value with the top two bits set (0xC0) to encode an offset into the DNS message where the remainder of the name continues. KTA's parser follows up to 10 pointer jumps before aborting with an error, preventing pointer-loop denial-of-service attacks against the kernel. This limit is deliberately conservative; RFC 1035 does not specify a limit, but 10 jumps is more than sufficient for any legitimate DNS response.

```
Key parsing constants: DNS_MAX_ANSWERS = 64 /* max answer records per response
*/ DNS_MAX_JUMPS = 10 /* max pointer compression jumps */ DNS_TYPE_A = 0x0001
/* IPv4 address record */ DNS_CLASS_IN = 0x0001 /* Internet class */
DNS_RDLENGTH_A
```

The route_store subsystem in route_store.c implements a custom protocol for multi-line writes from the Python daemon to /proc/traffic_analyzer_routes. The challenge is that a single write() syscall from Python may contain only a partial DEST/HOP/STATUS sequence, as the kernel's
/proc write buffer is limited to PAGE_SIZE (4 KB) per call.

The solution is the pending_write_ip module variable, which persists the current destination IP across separate write() syscalls within the same write session. The protocol is: (1) write DEST
{ip}\n to set the current destination; (2) write HOP {n}|{ip}|...fields...\n for each hop; (3) write STATUS {DONE|FAILED}\n to finalise the entry. The single ta_write() handler in proc_interface.c dispatches each line to the appropriate handler based on the first token.

```
Route write protocol example: DEST 8.8.8.8\n HOP 1|192.168.1.1||
1.2|Bhopal|IN|IN|23250000|77410000|0|ISP-Local\n HOP
2|103.47.144.1||8.5|Mumbai|IN|IN|19076000|72877000|45609|BSNL\n ... (additional
hops) ... STATUS DONE\n
```

ProcReader.cpp reads all /proc files and parses pipe-delimited lines into C++ data

structures. A key subtlety is the make_canonical() normalisation in the kernel module: because the kernel stores the lower IP in the src field regardless of direction, ProcReader must apply the inverse

transformation when displaying connections.

The rule is: if the connection's SRC_IP matches the local machine's IP, the connection is outgoing and the display order is correct. If SRC_IP does not match the local IP, the connection is incoming and the columns should be displayed with src/dst swapped. This logic is implemented in ProcReader::normaliseDirection() which is called for each row during parsing.

SparklineDelegate renders a miniature line chart for the RATE_OUT_BPS and RATE_IN_BPS columns. It maintains a 30-sample ring buffer per connection entry, appending the current rate on each 1-second refresh. The delegate must NOT have Q_OBJECT because it inherits from QStyledItemDelegate which already has a virtual table; adding Q_OBJECT creates a vtable conflict that manifests as a linker error.

Netlink messages are emitted from netlink_comm.c using nlmsg_new() and genlmsg_put() with GFP_ATOMIC (since emission can occur from softirq context during stats_update). The emission function locks the netlink send mutex, constructs the message, and calls netlink_broadcast() to multicast to all subscribers on the relevant group. Message construction uses fixed-size binary structs (ta_msg_conn, ta_msg_anomaly, ta_msg_dns, ta_msg_route) rather than TLV encoding for simplicity and performance.

The throttle logic is implemented as a per-entry timer: last_netlink_sent (ktime_t) and last_rate_out_bps, last_rate_in_bps fields in traffic_entry. On each stats_update call, the function computes the rate change ratio and compares with the 5-second forced interval threshold before deciding whether to emit. This reduces netlink traffic by approximately 95% compared to emitting on every packet, as measured during testing with iperf3 generating high-rate traffic.

CHAPTER 7 System Testing and Validation

This chapter describes the testing strategy for KTA, including unit tests, integration tests, stress tests, and edge case analysis. Testing was conducted on the same Ubuntu 22.04 / kernel 6.8.0 system described in Chapter 4.

Kernel module unit testing presents unique challenges: standard userspace testing frameworks (JUnit, pytest, Google Test) cannot be used directly in kernel context. KTA uses two approaches for unit-level validation.

The first approach is a userspace shim: key algorithmic functions are compiled into a standalone C program that exercises them with synthetic inputs and checks outputs. This approach was used for dns_parser.c (fed raw binary DNS response payloads), make_canonical() (checked normalisation of 100 flow pairs), and the anomaly threshold logic (checked boundary conditions at exactly the threshold values).

The second approach is in-kernel assertion macros: WARN_ON() calls at key invariant points (e.g., asserting that the connection table never exceeds 2,048 entries, that spin_lock is never called from irq context) generate kernel warnings in dmesg when violated. These assertions remain active in the released build as a lightweight runtime verification mechanism.

Test Module	Test Cases	Method	Pass Rate
dns_parser.c	48 test inputs (valid/malformed/com	Uprseesrsepda)c e shim	100%
make_canonical()	100 flow pairs (random + boundary	)Userspace shim	100%
Anomaly thresholds	25 boundary conditions per anoma	lyUtsyeprespace shim	100%
route_store protocol	20 write sequences (valid/partial/co	nUcsuerrsepnat)ce shim	100%
Cache eviction logic	10 table-full scenarios	In-kernel WARN_ON	100%

Table 7.1 — Unit test results summary

Integration tests verify that the three layers (kernel module, route daemon, Qt6 GUI) work correctly together end-to-end.

Test Scenario	Expected Outcome	Result
Load module, open GUI, browse to	GgoUoIgsleh.ocwoms Chrome process with DNS	dPoamssain populated
Kill and restart route daemon while	GRUoul tiessorpeesnume populating after daemo	nParessstart; no GUI
Open 2,000 parallel connections (n	cC) onnection table fills to 2,048; oldest	clPosaesds entries evic
Run nmap port scan; verify AlertPo	pAulpertPopup appears within 2 s; TrayIco	nPabsasdge = 1
Apply iptables block via GUI; verify	cBoloncnkeecdtivpitryocloesss's connections time ou	tP; abslosck rule visibl
Apply tc throttle via GUI; measure	eifpfeecrtfi3vetorathterottled process shows rate	Pcaosnsfigured limit

cras

h ted

e in iptables -L

Table 7.2 — Integration test results

Stress tests evaluate KTA's behaviour under high load conditions beyond normal operating parameters.

Test	Load	Duration	Observation
High packet rate (iperf3	lo1o0p0bMacbkp)s synthetic traff	ic10 min	CPU overhead < 0.05%; no dropped packets; pdate latency stats_u
High connection rate	1,000 new TCP conns/s	e5c m(ciunstom scr	i Cp to) nnection table reached peak 1,980 entries; evictio triggered corr
Concurrent DNS flood	10,000 DNS responses/	s1ecm(icnrafted pa	ydlnosa_dms)a p fill rate matched insert rate; no kernel panic 3 malformed s during restar
Route daemon restart io	oKpill/restart every 5 secon	d3s0 fmorin30	No route_store corruption; GUI displayed partial rout usage; failures

Table 7.3 — Stress test results

The following edge cases were specifically designed and tested to verify correctness of the most sensitive code paths:

- **DNS pointer loop:** A crafted DNS response with circular pointer compression (pointer A pointer B pointer A) was fed to the kernel hook. The 10-jump guard correctly terminated parsing and logged a warning to dmesg without crashing.
- **RST packet on CLOSED connection:** A TCP RST packet arriving for an already-CLOSED connection entry triggered the CLOSED CLOSED transition with no effect on the entry. No double-free or invalid state.
- **PID 0 (kernel thread) attribution:** Packets generated by kernel threads (swapper, kworker) have no userspace PID. KTA correctly stores PID=0 and labels these as "[kernel]" in the GUI.
- **Connection table full + new connection:** When the 2,048 entry limit was reached, new connections caused the oldest CLOSED entry to be evicted. Eviction correctly released memory and removed the entry from all caches.
- **Concurrent route writes:** Two simultaneous Python daemon instances (accidentally started) both attempted to write to /proc/traffic_analyzer_routes. The write mutex in proc_interface.c serialised the writes correctly; the second write successfully completed without data corruption.

References

[1] V. Jacobson, C. Leres, and S. McCanne, "tcpdump," LBNL Network Research Group, 1989. [Online]. Available: https://www.tcpdump.org

[2] G. Combs, "Wireshark Network Protocol Analyzer," 1998. [Online]. Available: https://www.wireshark.org

[3] R. Russell, "Linux Netfilter Hacking HOWTO," Netfilter Core Team, 1999. [Online]. Available: https://www.netfilter.org

[4] V. Jacobson, "Traceroute," Lawrence Berkeley National Laboratory, 1987. [Online]. Available: https://traceroute.sourceforge.net

[5] S. McCanne and V. Jacobson, "The BSD Packet Filter: A New Architecture for User-level Packet Capture," in Proc. USENIX Winter Technical Conference, San Diego, CA, 1993, pp. 259–269.

[6] B. Gregg, BPF Performance Tools: Linux System and Application Observability. Addison-Wesley Professional, 2019, ISBN 978-0-13-655482-1.

[7] D. E. Denning, "An Intrusion-Detection Model," IEEE Transactions on Software Engineering, vol. SE-13, no. 2, pp. 222–232, Feb. 1987.

[8] S. Staniford, J. A. Hoagland, and J. M. McAlerney, "Practical Automated Detection of Stealthy Portscans," Journal of Computer Security, vol. 10, no. 1–2, pp. 105–136, 2002.

[9] D. J. Bernstein, "SYN Cookies," 1996. [Online]. Available: http://cr.yp.to/syncookies.html

[10] Cisco Systems, "NetFlow Services Solutions Guide," Cisco IOS Release 12.3, 2004.

[11] F. Weimer, "Passive DNS Replication," in Proc. FIRST Conference on Computer Security Incident Handling, Singapore, 2005.

[12] M. Lad, D. Massey, D. Pei, Y. Wu, B. Zhang, and L. Zhang, "PHAS: A Prefix Hijack Alert System," in Proc. USENIX Security Symposium, Vancouver, BC, Canada, 2006.

[13] IETF, RFC 7011 — Specification of the IP Flow Information Export (IPFIX) Protocol, 2013.

[14] IETF, RFC 6480 — An Infrastructure to Support Secure Internet Routing (RPKI), 2012.

[15] P. Mockapetris, RFC 1035 — Domain Names — Implementation and Specification, IETF, 1987.

[16] MaxMind Inc., "GeoLite2 Free Geolocation Data," 2002–2025. [Online]. Available: https://dev.maxmind.com/geoip/geolite2-free-geolocation-data

[17] The Qt Company, "Qt 6 Documentation," 2021–2025. [Online]. Available: https://doc.qt.io/qt-6/

[18] Linux Kernel Community, "Netfilter Documentation," kernel.org, 2024. [Online]. Available: https://www.kernel.org/doc/html/latest/networking/netfilter.html

[19] J. Salmela, "Pi-hole: A Black Hole for Internet Advertisements," GitHub, 2014. [Online]. Available: https://github.com/pi-hole/pi-hole

[20] A. K. Reddy, "Kernel Traffic Analyzer v7.0 — Source Code Repository," GitHub, 2025. [Online]. Available: https://github.com/Anilkumar64/kernal-traffic-analyzer

[21] F. Y. Liu, K. M. Ting, and Z. H. Zhou, "Isolation Forest," in Proc. IEEE 8th Int. Conf. Data Mining (ICDM), Pisa, Italy, 2008, pp. 413–422.

[22] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection," in Proc. NDSS Symposium, San Diego, CA, 2018.

[23] A. V. Engelen, "nethogs: Per-process Bandwidth Monitoring," GitHub, 2004. [Online]. Available: https://github.com/raboof/nethogs

[24] P. Warren, "iftop: Display Bandwidth Usage on an Interface," 2002.

[Online]. Available: http://www.ex-parrot.com/~pdw/iftop/

[25] D. Taht and J. Morton, "RFC 8290 — The Flow Queue CoDel Packet Scheduler and AQM Algorithm," IETF, 2018.

[26] Linux Kernel Community, "Linux Kernel Documentation — workqueue," kernel.org, 2024. [Online]. Available: https://www.kernel.org/doc/html/latest/core-api/workqueue.html

[27] M. McKusick and G. Neville-Neil, The Design and Implementation of the FreeBSD Operating System, 2nd ed. Pearson Education, 2004.

[28] R. Love, Linux Kernel Development, 3rd ed. Addison-Wesley Professional, 2010, ISBN 978-0-672-32946-3.

[29] Verizon, "2023 Data Breach Investigations Report," Verizon Business, 2023. [Online]. Available: https://www.verizon.com/business/resources/reports/dbir/

[30] IBM Security, "Cost of a Data Breach Report 2023," IBM Corporation, 2023. [Online]. Available: https://www.ibm.com/reports/data-breach

[31] T. Bird, "Netfilter's Connection Tracking System," netfilter.org technical paper, 2004.

[32] IETF, RFC 4033 — DNS Security Introduction and Requirements (DNSSEC), 2005.